Software Quality Metrics Analysis of Open Source Projects

A Study of LangChain and Intel XPU Backend for Triton

Garrett Gruss

Southern Methodist University

December 4, 2025

Outline

Background

- Software quality metrics are essential for data-driven decisions in modern development
- Internal metrics (complexity, LOC) predict external quality (reliability, maintainability)
- DORA metrics have become industry standards for DevOps performance
- Open source projects need systematic quality assessment

Key Challenge: How do we measure quality in projects we don't control?

Motivation & Objectives

Motivation

- Systematically assess software quality
- Identify best practices
- Establish reusable benchmarks
- Provide actionable insights

Objectives

- Collect quality metrics over one month
- Analyze project health
- Compare across projects
- Generate quantitative insights

Selected Projects

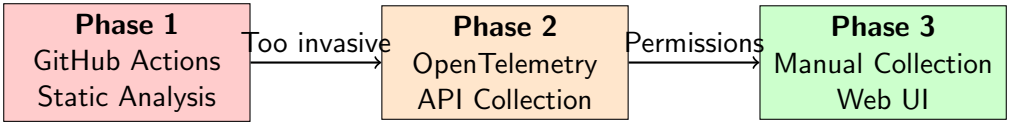
LangChain

- Framework for LLM applications
- Built in Python
- 128k stars, 20k forks
- 3,812 active contributors
- Analysis period: Oct 29 - Nov 27, 2025

Intel XPU Backend for Triton

- Compiler backend for Intel GPUs
- Out-of-tree Triton module
- 222 stars, 78 forks
- 538 active contributors
- Analysis period: Nov 4 - Dec 2, 2025

Methodology: Three Phases



Key Lesson: Methodological flexibility is essential when external constraints arise

Phase 1: GitHub Actions (Abandoned)

Planned Metrics

- Lines of Code (LOC)
- Cyclomatic Complexity
- Code Coverage
- Technical Debt
- Maintainability Index

Why Abandoned?

- Required repository modification
- Needed maintainer approval
- Risk of workflow conflicts
- Per-language customization
- Too invasive for external research

Phase 2: OpenTelemetry (Blocked)

Architecture

- OTel Collector (60s polling)
- GitHub GraphQL/REST APIs
- OpenObserve storage
- Docker Compose deployment

Target: 9 VCS Metrics

- vcs.change.count
- vcs.change.time_to_merge
- vcs.ref.lines_delta
- ...and more

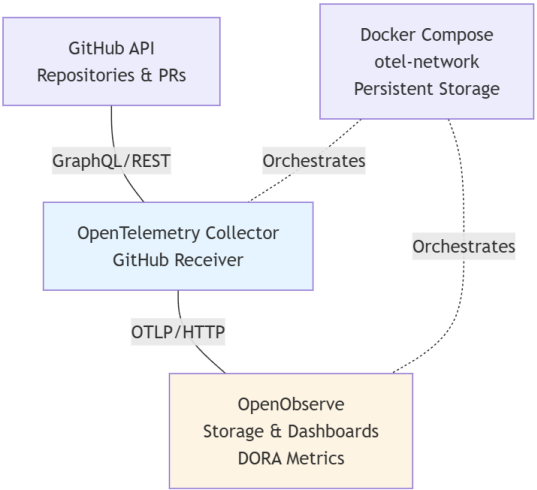


Figure: OpenTelemetry Architecture

Phase 3: Manual Collection (Final Approach)

Data Sources

- GitHub Insights tab (publicly accessible)
- Pull Requests page (filtered by date)
- Issues page (keyword search)
- Organized into CSV format

Advantages

- No authentication required
- No organizational approvals needed
- Reproducible by any researcher
- Sufficient granularity for DORA metrics

DORA Metrics: The Four Keys

1. **Deployment Frequency (DF)**
   - How often code is deployed to production
   - Proxy: Merged PR rate
2. **Lead Time for Changes (LT)**
   - Time from commit to production
   - Measured: PR open-to-merge duration
3. **Change Failure Rate (CFR)**
   - Percentage of deployments causing failures
   - Calculated: Bug issues / Total deployments
4. **Time to Restore Service (MTTR)**
   - Recovery time from failures
   - Measured: Bug report to fix merge

DORA Performance Classifications

Metric	Elite	High	Medium	Low
Deployment Freq.	Multiple/day	Weekly	Monthly	$<$ Monthly
Lead Time	$<$ 1 day	1-7 days	1-4 weeks	$>$ 1 month
Change Failure	0-15%	16-30%	31-45%	$>$ 45%
Time to Restore	$<$ 1 hour	$<$ 1 day	1-7 days	$>$ 1 week

Table: 2023 DORA State of DevOps Report Classifications

LangChain: Data Overview

Analysis Period: October 29 - November 27, 2025 (30 days)

Pull Requests

- 179 merged PRs
- 67 opened PRs
- 89 closed PRs

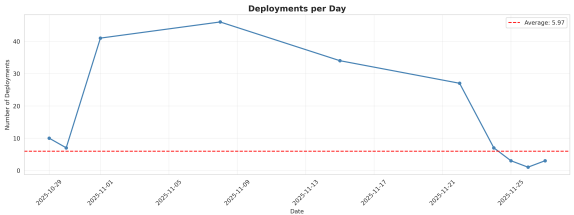
Issues

- 69 total issues
- 16 bug-related (23.2%)

LangChain: Deployment Frequency

Metrics

- Mean: 5.97 deployments/day
- Median: 8.5 deployments/day
- Weekly: 41.77 deployments
- Maximum: 46 in one day



Classification: ELITE

Multiple deployments per day indicates mature CI/CD pipeline

LangChain: Lead Time & Change Failure Rate

Lead Time for Changes

- Median: 6.00 days
- Mean: 4.98 days
- 25th percentile: 2.00 days
- 90th percentile: 7.00 days
- 5 PRs merged within 24 hours

Classification: HIGH

Efficient code review process

Change Failure Rate

- 16 bug issues / 179 deployments
- CFR: 8.94%
- Well below 15% threshold

Classification: ELITE

Excellent code quality and testing practices

LangChain: Summary

Metric	Value	Classification
Deployment Frequency	5.97/day, 41.77/week	Elite
Lead Time for Changes	6.00 days (median)	High
Change Failure Rate	8.94%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	**High Performer**	

Intel XPU: Data Overview

Analysis Period: November 4 - December 2, 2025 (29 days)

Pull Requests

- 104 merged PRs
- 17 opened PRs
- 73 closed PRs

Issues

- 42 total issues
- 11 bug-related (26.2%)

Intel XPU: Deployment Frequency & Change Failure Rate

Deployment Frequency

- Mean: 3.59 deployments/day
- Median: 9.0 deployments/day
- Weekly: 25.10 deployments
- Maximum: 28 in one day

Classification: ELITE

Multiple deployments per day for compiler backend

Change Failure Rate

- 11 bug issues / 104 deployments
- CFR: 10.58%
- Below 15% threshold

Classification: ELITE

Strong quality controls for complex domain

Intel XPU: Summary

Metric	Value	Classification
Deployment Frequency	3.59/day, 25.10/week	Elite
Lead Time for Changes	N/A (insufficient data)	N/A
Change Failure Rate	10.58%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	**Elite Performer***	

* Based on elite performance in deployment frequency and change failure rate

Comparative Analysis

Metric	LangChain	Intel XPU
Deployment Frequency	5.97/day (Elite)	3.59/day (Elite)
Lead Time for Changes	6.00 days (High)	N/A
Change Failure Rate	8.94% (Elite)	10.58% (Elite)
Time to Restore Service	N/A	N/A
Overall Rating	High Performer	Elite Performer

Key Insights:

- Both projects show elite deployment frequency
- Both maintain excellent change failure rates (¡15%)
- LangChain has measurable lead time in high performance range

Key Findings

Development Velocity

Both projects demonstrate elite-level deployment frequency, indicating mature CI/CD pipelines and rapid iteration capabilities

Code Quality

Change failure rates below 15% suggest:

- Comprehensive test coverage
- Effective code review processes
- Strong quality gates

Data Limitations

- MTTR could not be calculated (no bug-fix pair linking)
- Intel XPU had insufficient lead time data (only 3 matched PRs)
- One-month window may not capture long-term trends

Lessons Learned: Technical Challenges

1. **Repository Access Tradeoffs**
   - Internal metrics require invasive modifications
   - External research needs non-invasive approaches
2. **API & Permission Constraints**
   - GitHub PAT requires organizational approval
   - Rate limits (5,000/hr auth, 60/hr unauth)
   - Broad permission scopes create gatekeeping
3. **Data Completeness vs. Accessibility**
   - Phase 1: Max richness, max invasiveness
   - Phase 2: Moderate richness, still blocked
   - Phase 3: Min richness, universal access

Lessons Learned: Best Practices

Best Practices for External Software Quality Research:

1. Start with minimal access assumptions
2. Prioritize non-invasive observation
3. Leverage public observability (GitHub Insights)
4. Use industry-standard frameworks (DORA, SPACE)
5. Automate analysis, not collection
6. Build incremental validation
7. Document methodological pivots

Value of DORA Metrics:

- Industry standardization enables benchmarking
- Publicly derivable from GitHub data
- Actionable elite/high/medium/low classifications

Future Work

Extended Data Collection

- 3-6 month analysis periods
- Additional open source projects
- Longitudinal trend analysis

Enhanced Metrics

- Code-level metrics (if access obtained)
- Contributor activity patterns
- Code review analytics

ML Applications

- Predict project health from trends
- NLP for better issue classification
- Anomaly detection in dev patterns

Tool Development

- Real-time DORA dashboard
- SaaS for OSS health analysis
- CI/CD pipeline integration

Conclusion

Summary

This study successfully analyzed software quality metrics for LangChain and Intel XPU Backend for Triton using DORA metrics derived from publicly accessible GitHub data.

Key Achievements

- Demonstrated adaptive methodology in face of technical constraints
- Validated DORA metrics as effective quality indicators
- Established reproducible approach for external research
- Provided actionable insights into project health

Impact

The methodology developed is reproducible and can be applied to any open source project with public GitHub repositories, enabling systematic quality assessment without requiring repository access.

Questions?

Garrett Gruss
Southern Methodist University

Software Quality Metrics Analysis of Open Source Projects